Distinguished Lecture Series I: Avi Wigderson, "The power and weakness of randomness in computation"

Terence Tao · 10 January 2008

https://terrytao.wordpress.com/2008/01/10/distinguished-lecture-series-i-avi-wigderson-the-power-and-weakness-of-randomness-in-computation/

tao2typ · converted to Typst 2026-10-14 (UTC)

The Distinguished Lecture Series at UCLA for this winter quarter is given by Avi Wigderson, who is lecturing on "some topics in computational complexity". In his first lecture on Wednesday, Avi gave a wonderful talk (in his inimitably entertaining style) on "The power and weakness of randomness in computation". The talk was based on these slides. He also gave a sort of "encore" on zero-knowledge proofs in more informal discussions after the main talk.

As always, any errors here are due to my transcription and interpretation.

The theme of this talk was the close relationship between *hardness* (computational complexity) and *randomness* (and specifically, the use of randomness to save on resources such as time or memory).

— Computational complexity —

Avi began with some examples to illustrate "easy" and "hard" computational problems. For instance:

1. *Multiplication* of two n-digit numbers is easy; the long multiplication algorithm one learns in primary school lets one do this in $O(n^2)$ steps (and faster algorithms are also known); but
2. *Factorisation* of an n-digit number is believed to be hard; for large n, the fastest known factoring algorithm is believed to take $\exp(O(n^{1/3} \log^{2/3} n))$ steps (the fastest algorithm with a rigorous costing takes $\exp(O(n^{1/2} \log^{1/2} n))$ steps). Avi made the point that all of us implicitly agree with this belief of hardness when we entrust modern cryptographic protocols to protect our information when, say, using a credit card over the internet.
3. *Theorem-proving* – the task of finding a proof of a given statement S using at most n steps in a given axiom system – is also widely believed to be hard (indeed, my job, and the job of my colleagues, arguably depends crucially on this belief :-)). [In contrast, *verifying* that a given n-step proof actually does prove S is (in principle) easy, though not always in practice...]

It is not known whether factoring is truly hard, or whether theorem-proving is truly hard. But there is a known reduction: if theorem-proving is easy (in the sense that it is a polynomial time algorithm), then factoring is also easy. This is because theorem-proving is an NP-complete problem (this follows from the Cook-Levin theorem).

The famous $P \neq NP$ problem is thus formally equivalent to the statement that theorem proving is not easy. At a more philosophical level, one can view $P \neq NP$ as an assertion that "creativity cannot be automated".

There are many other NP-complete problems known in mathematics and science (indeed, Avi noted that such problems appear to be "uniformly distributed" throughout these disciplines). We thus have the following informal

> **Belief 1**: "natural" problems such as factoring, theorem-proving, 3-colourability of graphs, computation of the permanent, etc. are hard.

(One can view the assertion $P \neq NP$ as a particularly famous instantiation of this belief, but there are others.)

— The power of randomness —

Avi now turned from hardness to randomness. There is a vast body of empirical evidence suggesting that probabilistic algorithms can be much faster than deterministic algorithms, at the cost of introducing a small probability that the algorithm can fail. In particular, there are many problems for which a polynomial time probabilistic algorithm is known, but all known deterministic algorithms require exponential time at least.

More formally, we say that a problem has a polynomial time probabilistic algorithm if there exists an algorithm that takes the given input, combines it with a polynomially long string of random independent unbiased bits (or "coin flips"), and after a polynomial number of computations returns an output which correctly solves the problem for at least 99% (say) of the possible values of the random bits. Note that there is nothing special about 99%; by iterating the algorithm repeatedly with the same input (but with fresh random bits) and then taking a majority vote, one can amplify the accuracy to the point where the failure rate is exponentially small while the run time remains polynomial.

All things being equal, of course, a deterministic algorithm would still be preferable to a probabilistic one: 100% success is better than 99% success. But probabilistic algorithms can be much faster, as the following four diverse examples show:

1. Primality testing. The question of whether one can quickly determine whether a given n-digit number is prime was considered as far back as Gauss (who mused whether there was an "indefatigable calculator" who could perform such tasks). Naive primality testing methods tend to be exponential time; the first polynomial time algorithms (such as Miller-Rabin and Solovay-Strassen, discovered in the late 1970s) were probabilistic. A polynomial time deterministic primality test was only recently discovered by Agrawal, Kayal, and Saxena (though, in practice, it is still a little slower than the probabilistic tests). On the other hand, if we modify the problem slightly to that of *finding* an n-digit prime for a given n, no polynomial time deterministic algorithm is known, although one can trivially iterate the probabilistic primality tests to obtain a polynomial time probabilistic prime-finding algorithm.
2. Verification of polynomial identities. Suppose one is given two polynomials $P(x_1, ..., x_n), Q(x_1, ..., x_n)$ of degree at most d in n variables, expressed in some easily computable formula (e.g. involving determinants). Can one quickly determine whether the two polynomials are equal? (A good example here is Vandermonde's identity). Expanding out all the coefficients can take exponential time. But there is a quick and simple probabilistic algorithm: simply test the identity $P(x_1, ..., x_n) = Q(x_1, ..., x_n)$ for some randomly chosen inputs $(x_1, ..., x_n) \in \{1, ..., 100d\}^n$; the fact that this algorithm is probabilistically sound (gives few false positives) follows from the Schwartz-Zippel lemma. (It is clear that the test gives no false negatives.) On the other hand, no sub-exponential deterministic algorithm for polynomial identity testing is known.
3. Large Fourier coefficients. Given an n-bit boolean function $f : \mathbb{F}_2^n \to \{-1, 1\}$ (expressed in some rapidly computable form) and a threshold $\varepsilon > 0$, a simple application of Plancherel's identity shows that there are at most $1/\varepsilon^2$ frequencies $\xi \in \mathbb{F}_2^n$ for which the Fourier coefficient $\hat{f}(\xi) := \frac{1}{2^n} \sum_{x \in \mathbb{F}_2^n} (-1)^{x \cdot \xi} f(x)$ has magnitude at least ε. But how can one find these frequencies? Even using a fast Fourier transform, a direct search would require exponential time in n; no deterministic subexponential time algorithm is known. However, the Goldreich-Levin theorem can be used to provide a probabilistic polynomial time algorithm for this task.
4. Computing volume. Given some effective description of a d-dimensional convex body (e.g. a list of bounding hyperplanes), can one quickly determine the volume? This problem is #P complete (thus, even harder than NP-complete!), thanks to work of Dyer-Frieze and Khachiyan, so the answer is almost certainly no. But what if one wanted to compute the volume up to a factor of 2? No fast deterministic algorithm is known, but by use of random walks in the polytope one can obtain a probabilistic polynomial time algorithm; this was first done by Dyer, Frieze, and Kannan.

Based on this empirical evidence, one is led to

> **Belief 2**: There are problems which have easy probabilistic algorithms, but no easy deterministic algorithms.

(This belief is often formalised as the assertion $P \neq BPP$.)

— The weakness of randomness —

Recent developments by Avi and his co-authors strongly suggest the following surprising fact:

Belief 1 and Belief 2 cannot simultaneously be true.

Morally, one now expects that if $P \neq NP$, then $P = BPP$, although the strongest rigorous result in this direction so far is that if a sufficiently difficult problem (3-SAT will do, or any problem in the class E) requires an exponentially large circuit to solve, then $P = BPP$ (this is a slight oversimplification; the precise result is in this paper of Impagliazzo and Wigderson). For a more complete survey, I can recommend Avi's ICM lecture article on these topics.

The underlying reason for this phenomenon is this: if computationally hard problems exist, then these problems can be used to create strong pseudorandom generators – streams of bits which can "fool" fast probabilistic algorithms in the sense that they are indistinguishable from genuinely random bits, even if they actually have much lower entropy. These generators are deterministic except for the choice of seed, but one can arrange the number of possible seeds to be polynomial rather than exponential (by requiring the length of the seed to be logarithmic). As such, one can then substitute the random bits in a probabilistic algorithm by the pseudorandom bits to *derandomise* the algorithm into a deterministic one with comparable run-time.

[As pointed out in the comments, the use of pseudorandom number generators is of course used heavily in real-life computers, since modern (classical) computers usually do not have access to high-quality streams of genuine randomness. Also, I should mention (and Avi noted also) that the computational complexity notion of pseudorandomness has some relation to the additive combinatorial notion of pseudorandomness that I refer to sometimes in my own lectures; here one is fooling an algorithm or computable function rather than a linear or polynomial phase, but the idea is basically the same.]

Avi showed just one key step in the process, in which k random bits were converted into k+1 pseudorandom bits, for a logarithmically small value of k (which is how long one can arrange the seed to be). The idea is to add a new bit which is equal to the output of a computationally hard problem with the preceding k bits as input. If one can ensure that this problem is in fact computationally hard for average-case inputs (and not just worst-case inputs, which is what hypotheses such as $P \neq NP$ provide), then this stream of bits cannot be easily distinguished from genuinely random bits. So one needs some *hardness amplification* tricks to convert worst-case difficult problems to average-case difficult problems.

— The power of randomness in other settings —

Avi then turned to some other advantages of randomness in algorithms, beyond that of saving time. Another is that of saving memory, which he illustrated with the classical maze exploration problem. Suppose one wanted to get from one vertex in a connected n-vertex graph (representing the maze) to another. If one is allowed to map out the graph, it is easy to achieve this deterministically in polynomial time using a polynomial amount of memory. But what if one has essentially no memory capacity (other than to remember which vertex is the destination vertex)? Then one can still reach the desired vertex in polynomial time ($O(n^2)$, in fact) with high probability by an utterly simple random algorithm, namely a random walk on the graph.

Two other famous, and striking, applications of randomness lie in interactive proof systems, in which a verifier is trying to check a claim that a prover can prove a given statement S (e.g. the Riemann hypothesis). More formally, the verifier employs some algorithm V that takes S as input, performs some

queries to the prover, and then returns a boolean output. A deterministic verifier should satisfy the following two properties:

1. Completeness. If the prover does indeed have a proof of S, then V should return "true".
2. Soundness. If the prover does not have a proof of S, then V should return "false".

A probabilistic verifier is defined similarly, except that if the prover does not have a proof of S, we only expect V to return "false" with probability at least 99% (say).

Constructing a verifier with these properties is not too difficult: the prover can simply hand over the entire proof to the verifier when asked, and the verifier can then check all the steps of the proof. Of course, if the proof has length n, then this procedure will take time O(n). But a remarkable fact is that a probabilistic verifier only needs to see a very tiny portion of the proof to verify it!

> **PCP theorem**. Every proof of a statement S of length n in a formal system can be converted (in polynomial time) into a proof which can be verified by a probabilistic verifier in O(log n) time using only O(1) queries to the prover.

In a rather different direction, instead of addressing the verifier's desire to verify a proof as quickly as possible, one can consider the prover's problem of wanting to have his or her claim of proving S verified without allowing the verifier to learn enough of the proof to publish it first. Another remarkable fact is that this is, in fact, possible (conditioning on a complexity hypothesis):

> **Existence of zero-knowledge proofs**. Suppose factoring is hard. There exists a polynomial time probabilistic verifier for proofs of a statement S of length n which is *zero-knowledge* in the sense that even in the absence of the prover (or of the proof), the verifier can simulate the distribution of answers that the prover would supply to the verifier's queries (given at least one proof in hand, of course); more informally, the verification algorithm does not tell the verifier anything that the verifier did not already know, other than that the statement S is provable.

This statement is rather unintuitive to comprehend, let alone accept, at first, but Avi then explained the basic idea using a graph colouring model.

— 3-colouring and zero-knowledge proofs —

The question of determining whether a given graph G with n edges is 3-colourable is known to be NP-complete (and thus just as hard as theorem proving). Because of this (and a little bit of extra work), it is possible to reduce matters to considering the problem of verifying a very specific type of claim: that a given graph G (which is known to both the verifier and prover) admits a 3-colouring.

Suppose that the prover has in fact managed to 3-colour the graph G (using, say, the colours red, green, and blue), and wants to convince the verifier of this fact, without letting the verifier learn anything about how to colour this graph himself or herself, or indeed to learn anything at all other than the fact that G is 3-colourable.

This can be done probabilistically by exploiting a simple symmetry in the space of all 3-colourings of G, namely the ability to permute the colours around and convert any given 3-colouring into one of 3! = 6 colourings. We use this to create the following verification algorithm:

1. The prover picks a 3-colouring of the graph using red, green, and blue, permuting the colours randomly if desired. The prover then *commits* to this colouring; if this were done physically, the prover could place the colour of each vertex in an envelope atop that vertex. To do it digitally, one needs a cryptographic commitment scheme, which can be set up provided that factoring is hard (other complexity assumptions can be substituted here).
2. The verifier then picks two adjacent vertices of the graph arbitrarily, and asks the prover to reveal the colours of the two selected vertices. If the two colours agree (or if a colour other than red, green, or blue appears), then the algorithm returns "false".

3. Otherwise, return to Step 1 and start over. If the algorithm does not return false in (say) n^2 iterations of this process, return "true".

Note that if the prover does not in fact know of any 3-colouring of G, then the verifier will be able to detect this fact with probability at least 1/n in any round of the above procedure, since by picking an edge at random one has at least a 1/n probability of picking an edge for which the prover's colouring fails to be a 3-colouring. So the algorithm is probabilistically sound. It is also obviously complete. So why is it zero knowledge? Because if the prover randomly shuffles the colours at each round, independently of all previous rounds, then for any given round, the two colours that the verifier gets to see are distributed completely at random among all six ordered pairs of distinct colours. This is a distribution of query responses that the verifier could have generated himself or herself, and so (provided that the prover really does have a colouring) this algorithm is not providing any new information that the verifier did not already have.

[*Update*, Jan 10: typos corrected, comments added.]

Comments (19)

 **otto** · 10 January 2008

Terry, it seems that Belief 2 would be formalized as P \neq BPP.

 **Will** · 10 January 2008

Hi Terry, thanks for posting your interpretations of lectures/seminars. They are a model of clarity.
A small typo : it should read Agrawal, Kay*al*, and Sax*ena*. Thanks.

 **roland** · 10 January 2008

SAT does not need exponential space. The possible assignments of a SAT instance can be tested one after the other reusing space. BBP=P if there is a language in E (exponantial time, linear exponent) that can't be solved in subexponential time even with some subexponential advice (the deciding machine processes the word and the advice which depends on the length of the word.)

 **Anonymous** · 10 January 2008

The verifier given by the PCP Theorem will require O(log n) random bits. Simply to read those bits, it will need O(log n) time, not O(1).

 **Allen Knutson** · 10 January 2008

Maybe you pointed out this out somewhere and I missed it: it seems worth reminding the reader that the idea of using a computationally subtle function to produce bits to then hand off to a randomized algorithm is, indeed, exactly how randomized algorithms are run on computers in practice.

 **Terence Tao** · 10 January 2008

Thanks for the corrections and suggestions!

 **Anonymous** · 11 January 2008

I didn't realize it was considered surprising that beliefs 1 and 2 can't be simultaneously true. I thought most theorists (certainly anyone who does cryptography) believes P=BPP because otherwise there would be no pseudo-random generators in NP.
Russell Impagliazzo has an interesting essay about different levels at which P=NP could be true or false. A summary and link is here:
http://weblog.fortnow.com/2004/06/impagliazzos-five-worlds.html

 **Anonymous** · 11 January 2008

Question about PCP theorem: suppose it turns out that the first occurrence of the ten-digit sequence 0123456789 in the decimal expansion of pi starts at the 37 trillionth digit. We could write the statement S:
statement S: FIRST_PI('0123456789') = N (where N = 37 trillion)
Obviously we can confirm S in poly(N) time by computing all those digits.
Does the PCP theorem really say that if we do enough work, we should be able to convince another person of S in O(log N) steps by answering O(1) queries that they send us?

 **Distinguished Lecture Series II: Avi Wigderson, "Expander graphs - constructions and applications" « What's new** · 11 January 2008

[…] with his second lecture "Expander Graphs – Constructions and Applications". As in the previous lecture, he spent some additional time after the talk on an "encore", which in this case was […]

 **Terence Tao** · 11 January 2008

Dear Anonymous 1,
I think Avi's point was that based purely on empirical evidence of one can actually do in polynomial time, it would seem that both $P \neq NP$ and $P \neq BPP$, but once one sees the connection between hardness and pseudorandom number generation one would be led instead to the presumably more correct position that $P \neq NP$ and $P = BPP$.
Dear Anonymous 2,
I am not an expert in PCP (apart from a conversation this morning with Avi), but I believe the answer is yes, that after computing N digits of pi and then doing poly(N) more work to encode that computation in a suitable format, that the resulting encoded computation can be verified probabilistically using only O(1) inspections of that code and O(log N) work.
An analogy would be with locally testable (and locally decodable) codes. There are error-correcting codes in which one can take an N-bit string and verify quickly (just by looking at O(1) of the bits) whether this string is indeed close to a codeword. In particular, those O(1) queries can be used as a "proof" that the prover possesses a codeword. If I understand correctly, the original proofs of the PCP theorem proceeded by showing that the assertion that one possesses a proof of a statement S can be converted (roughly speaking) after a polynomial amount of work into an assertion that one possesses a codeword of a certain locally testable error-correcting code. (There is a later and simpler proof of Dinur which proceeds in a slightly different manner, combining error correcting codes with expander graphs in a clever way.)

 **Anonymous** · 11 January 2008

Thanks. This theorem sounds amazing and I'm going to have to try to learn something about it.
(Note: anonymouses 1 and 2 and 3 (me) are all the same person).

 **Johan Richter** · 13 January 2008

There is an explanation of the PCP theorem and its proof in volume 44, number 1 of the Bulletin of the AMS. http://www.ams.org/bull/2007-44-01/S0273-0979-06-01143-8/S0273-0979-06-01143-8.pdf

 **Anonymous** · 15 January 2008

i was wondering about the suggestive statement 'creativity cannot be automated' in the context of quantum computation, for example theorem proving. i have heard a characterization of modern computers as built largely on newtonian principles and hence capable of solving problems of newtonian complexity: iterative ode/pde etc. but quantum computers built on quantum principles can process information at the quantum level, hence solve schroedinger eqn at the atomic level across an entire material or simultaneously calculate on all numbers etc.
it doesnt seem implausible that a quantum computer might be able to do theorem proving and exhibit 'creativity' in the sense of this blog entry, does this have any philosophical or scientific implications? for instance, if a quantum computer can do theorem proving, then should we not consider theorem proving essentially 'creative'?

 **Terence Tao** · 15 January 2008

Dear Anonymous,
It is currently not known whether quantum computers are powerful enough to solve NP-complete problems in polynomial time. (I believe the name for this assertion is $NP \subset BQP$.) Quantum computing has some aspects of parallelism but it is not as strong as a massively parallel classical computer, due to the sharply limited number of operations one is permitted to perform on a superposition of states.

This article by Scott Aaronson (who, incidentally, states the above point in the very title of his blog) has more discussion on this topic:

> Why are solitons stable?

 **P=NP, relativisation, and multiple choice exams « What's new** · 1 August 2009

[...] of time, but are difficult to distinguish from genuinely random strings by any quick tests. See my writeup of this lecture by Avi Wigderson for more [...]

 **PneqBPP** · 6 May 2017

'Morally, one now expects that if P \neq NP, then P = BPP'. This statement cannot be right since contrapositive is false.

 **Terence Tao** · 6 May 2017

The equivalence of a statement with its contrapositive is always valid when working with deductive logic, but is not always accurate when working with non-deductive forms of reasoning, such as inductive logic. A classic example of this distinction arises in the raven paradox: https://en.wikipedia.org/wiki/Raven_paradox

IdeaGraph's POCO vis-a-vis Blockchain's POW – Crypto 3.0 – IdeaGraph · 31 October 2018

[...] source: https://terrytao.wordpress.com/2008/01/10/distinguished-lecture-series-i-avi-wigderson-the-power-and... [...]

 **Anonymous** · 28 January 2024

An $O(n\log(n))$ multitape machine exists to multiply two n-bit integers.Every circuit multiplying two n-bit integers must have size $\Omega(n\log(n))$ unless some conjecture is false.